

The final assessment for this module consists of 5 Questions. Please read each of the questions carefully and you should attempt **ALL** of the questions. The marking rubric can be seen on the assessment page on Canvas.

Submission Requirements

- Files should be submitted in the .ipynb format
- Submit either one file per question (5 files total) or one single file containing all answers

1) You are given the following uncommented code for function foo:

```
def foo(ts):  
    return [t * 2 for t in ts if t[0].lower() in "aeiou"]
```

- i) Provide a commented version of the code above. Using a docstring explain the purpose of this piece of code.
- ii) Rewrite the code using a for loop instead of list comprehension, maintaining the same functionality. Using a docstring explain the purpose of this piece of code. Provide clear and concise comments explaining the function of the code written.

2)

- i) Write a function that asks the user for 5 unique strings one at a time using a For loop and stores them in a set. The function must return a set with 5 values.
- ii) Write a function that accepts a set 'words' and an integer 'cutoff' and returns a set containing the elements of 'words' that are shortened to be no longer than the 'cutoff'.
- iii) Write code to run the functions in part i) and ii), with '5' as the 'cutoff' and user's input as 'words'. The result should be printed to the console.
- iv) For parts i), ii) and iii) write efficient code and document it as detailed in the Documentation Requirements.

3)

- i) Modify the function in Appendix B to read integers separated by spaces from the text file and store them as a list of integers.
- ii) Extend the function in part i) to also use the insertion sort algorithm to sort the list in descending order.
- iii) Write code to run the function written in part ii) with the filename being 'input.txt'. Assume that the integers are separated by spaces, but can be on different lines. You can use the following contents of the 'input.txt' file to test your code:

```
1 7 13 2 57  
18 6 21  
  
33 22 15
```

- iv) For parts i), ii), iii) and iv) write efficient code and document it as detailed in the Documentation Requirements.

4) Your manager gives you the following task:

Starter	Main	Desert
Calamari rings 6.25	Grilled Salmon 12.60	Ice Cream 3.60
Chicken Wings 6.50	Beef Burger 11.50	Lemon Tart 4.75
Grilled Halloumi 5.50	Vegetable Lasagne 51	Cheesecake 5.25

- i) Write a program that has three dictionaries as shown in a table above. In each dictionary, the key should be the item's name and the value should be the price.
 - ii) Now create a list called 'menu'. This list should have the two dictionaries from part i) as its elements.
 - iii) Write a function that takes a list of dictionaries as an argument and ask the user to input a string. The function should then find and return the price of that dish in the menu. The function should use a nested for loop to do the processing. If the dish name is not found in the menu, then the appropriate message should be displayed in the console and 'None' should be returned by the function.
 - iv) Write code to run the function written in part iii) using the 'menu' list created in part ii). The result should be printed to the console.
 - v) For parts i), ii), iii) and iv) write efficient code and document it as detailed in the Documentation Requirements.
- 5) Your manager now needs you to provide a more complex version of the code that you have written in question 4. Now the code should:
- i) Create the list called 'menu' like in Q4 parts i) and ii). Then modify it to be a dictionary instead of a list. Write a function to print the contents of the 'menu' dictionary to the console in an organised and easy to read format.
 - ii) Write a function called 'add_dish' that has three parameters: 'course', 'name' and 'price'. The function should then add the new entry to a dictionary in 'menu' specified by 'course' parameter. The function should let the user know that the item has been added and print the updated 'items' to the console. If the 'course' parameter is invalid, then an appropriate message should be printed to the console.
 - iii) Write a new function that uses the function from Q4 part iii) to ask the user to enter three values in the console: one choice for each course. The function should then calculate and return the total amount for the chosen meal and print the appropriate message to the console. If the user enters a missing name or chooses multiple of the same course, then the message should be printed to the console letting the user know that an invalid selection was made and return 'None'.
 - iv) Write code to run the functions written in parts i), ii) and iii) using the 'menu' dictionary created in part i). The results should be printed to the console.
 - v) For parts i), ii), iii) and iv) write efficient code and document it as detailed in the Documentation Requirements.

Documentation Requirements

- Comments and docstrings should be concise
- Comments and docstrings should have a consistent format throughout the solutions
- Comments should explain the function of the code written
- Each function must include a **docstring** that explains the purpose of the function, parameters and the return value
- Variable and function names must be descriptive and follow a consistent naming style, but maximum 25 characters

Appendix A

```
def ask_input(name):  
    my_string = input("{} , please enter a string: ".format(name))  
    return my_string
```

Appendix B

```
def read_file(filename):  
    file = open(filename, 'r')  
    contents = file.read()  
    file.close()  
    return contents
```